Programming with Python



Outline

Day1

1. (a) Introduction to Python and Basics

   (b) Flow Control

Day2

2. Strings
3. Tuples
4. Dictionaries

Day3

5. Lists
6. Functions (Part1)

Outline

Day 4

6. Functions (Part2)
7. File Handling

Day5

8. OOPs in Python
9. Regular Expressions

Day6

10. Exception Handling
11. Python Libraries - Intro

Outline

Day7

12. Python Packages for Data Mining and Machine Learning -
    a) Numpy
    b) Pandas
    c) Visualization

What is Python?

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics.



Designed By: **Guido van Rossum**

Name came from a 1970s British television show: *Monty* ***Python's*** *Flying Circus*



First released in 1990

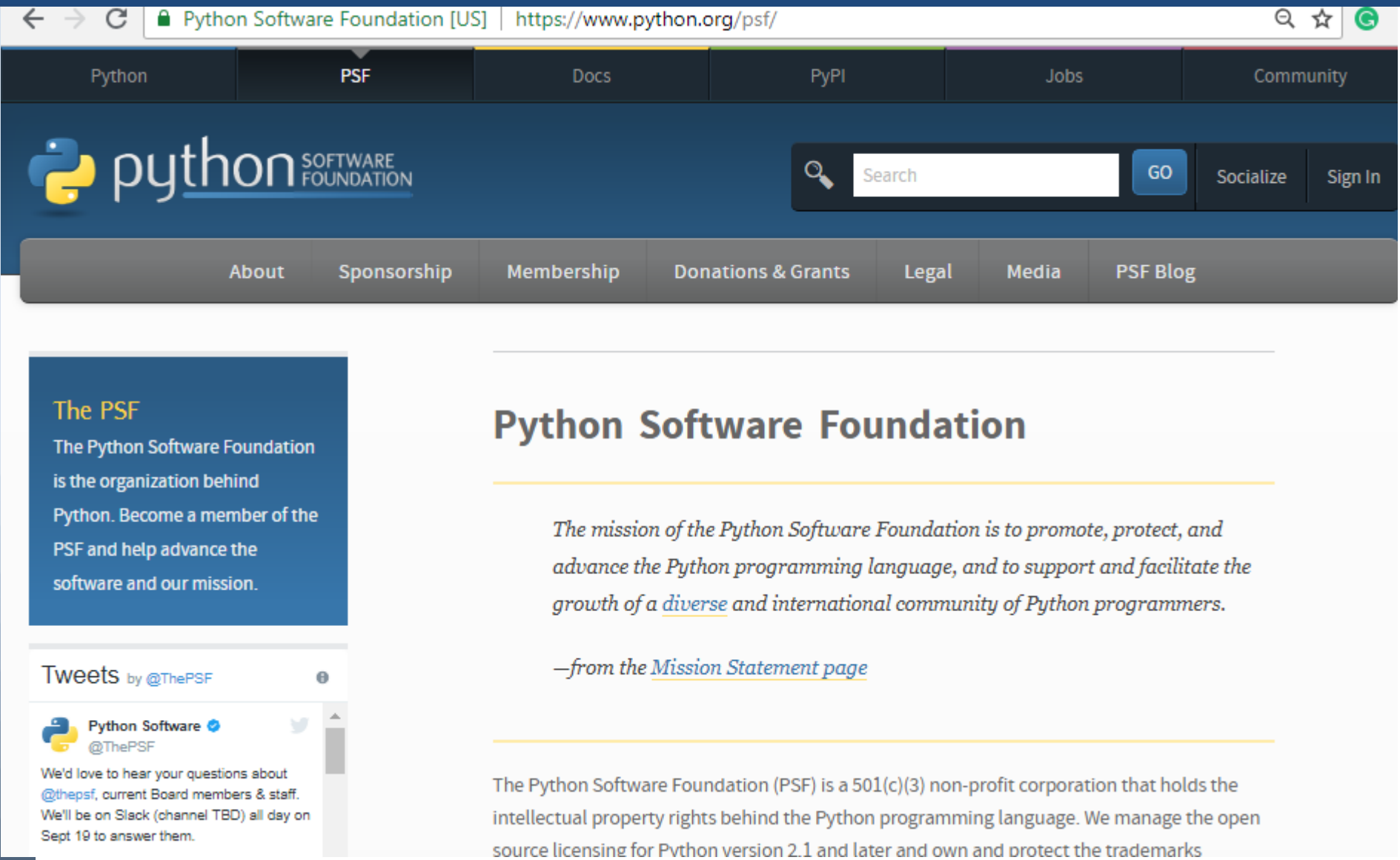
https://en.wikipedia.org/wiki/Python_(programming_language)

Reference: https://www.python.org/~guido/

What is Python?

- Multi Functional: (General Purpose)
  - **Procedural**
  - **Object-orientated**
- Computer Programming for everybody
  - An **easy** and intuitive language just as powerful as major competitors
  - **Open source**, so anyone can contribute to its development
  - Code that is as **understandable** as plain English
  - Suitability for everyday tasks, **allowing for short development times**
  - **Extensible:** Reusable code using modules and packages, Easy to write new modules in C..

How is Python developed and Supported?



How is Python developed and Supported?

- A popular **open source** system, has very active development community.
- **Developed by: The *PSF* (Python Software Foundation),** a formal nonprofit group, organizes conferences and deals with intellectual property issues.
- **License:** PSF License
- **PEP** – Python Enhancement proposals

PEP

- https://www.python.org/dev/peps/
- This PEP contains the index of all **Python Enhancement Proposals**, known as PEPs.
- PEP numbers are assigned by the PEP editors, and once assigned are never changed.

PEP

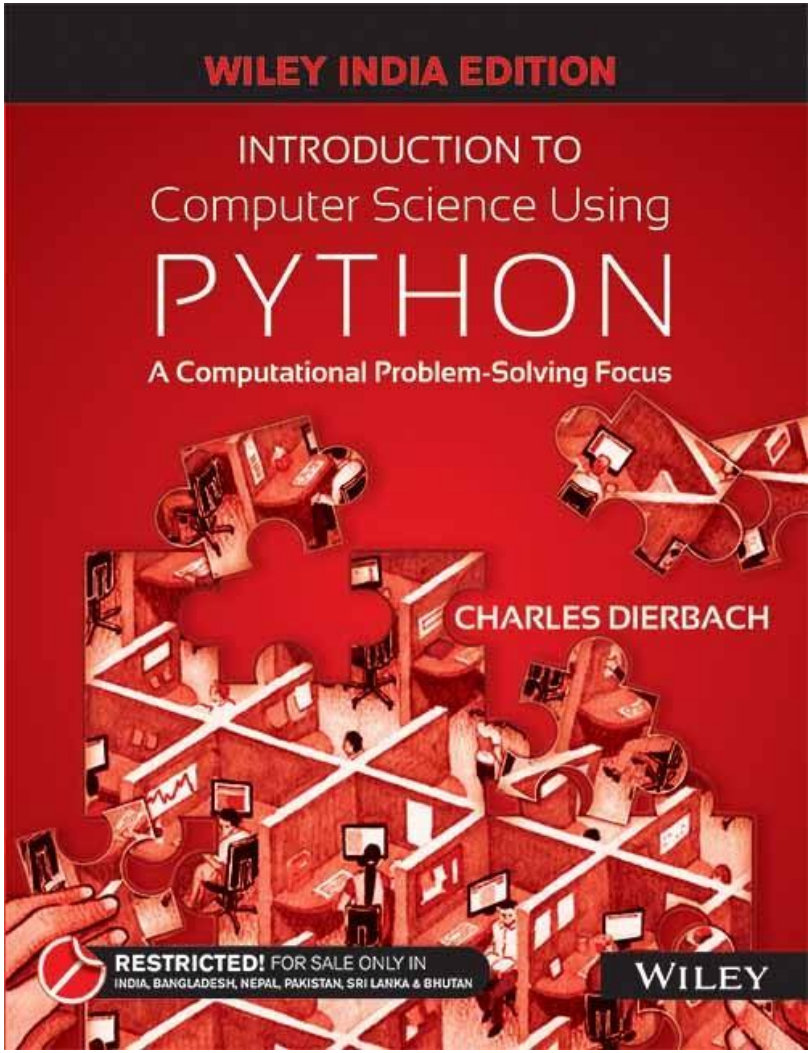
Python Software Foundation [US] | https://www.python.org/dev/peps/#finished-peps-done-with-a-stable-interface

P	8001	Python Governance Voting Process	Warsaw

Other Informational PEPs

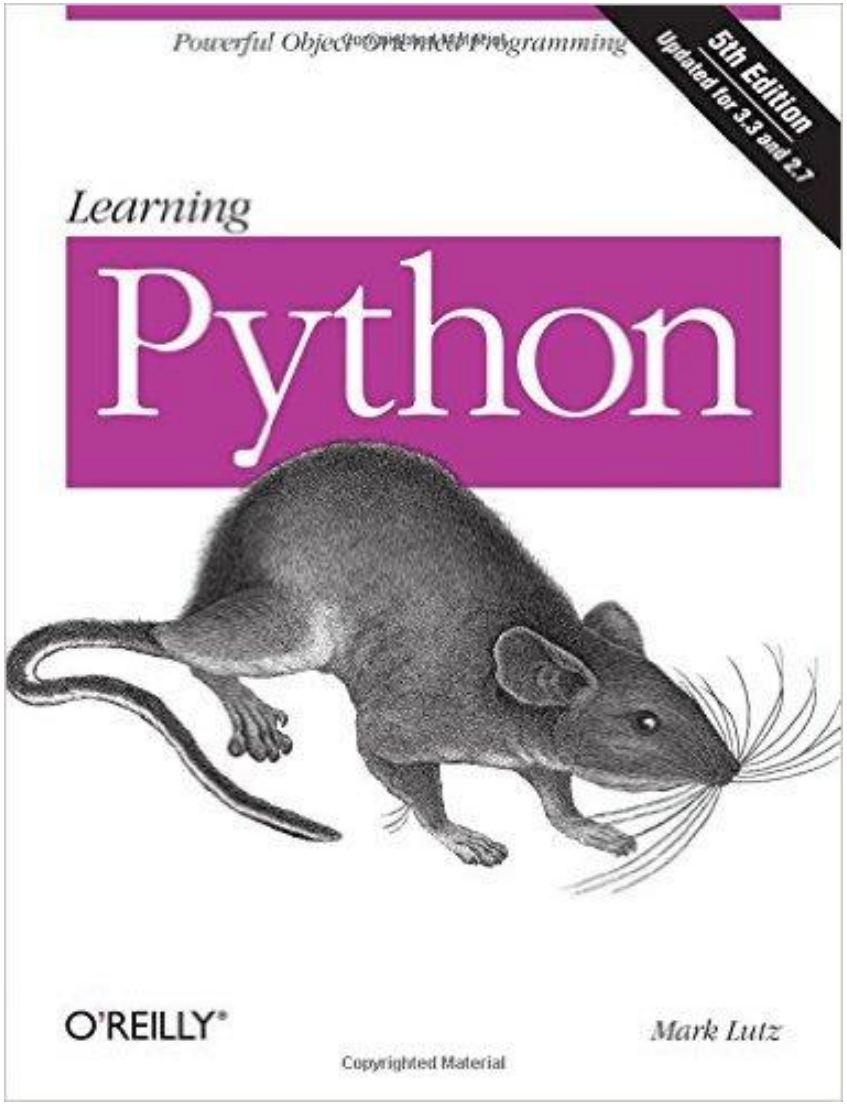
	PEP	PEP Title	PEP Author(s)
I	13	Python Language Governance	Warsaw
I	20	The Zen of Python	Peters
I	101	Doing Python Releases 101	Warsaw, GvR
IF	247	API for Cryptographic Hash Functions	Kuchling
IF	248	Python Database API Specification v1.0	Lemburg
IF	249	Python Database API Specification v2.0	Lemburg
I	257	Docstring Conventions	Goodger, GvR
IF	272	API for Block Encryption Algorithms v1.0	Kuchling
I	287	reStructuredText Docstring Format	Goodger
I	290	Code Migration and Modernization	Hettinger
IF	291	Backward Compatibility for the Python 2 Standard ...	Norwitz
IF	333	Python Web Server Gateway Interface v1.0	Eby
I	373	Python 2.7 Release Schedule	Peterson

Python Material (Level 1)



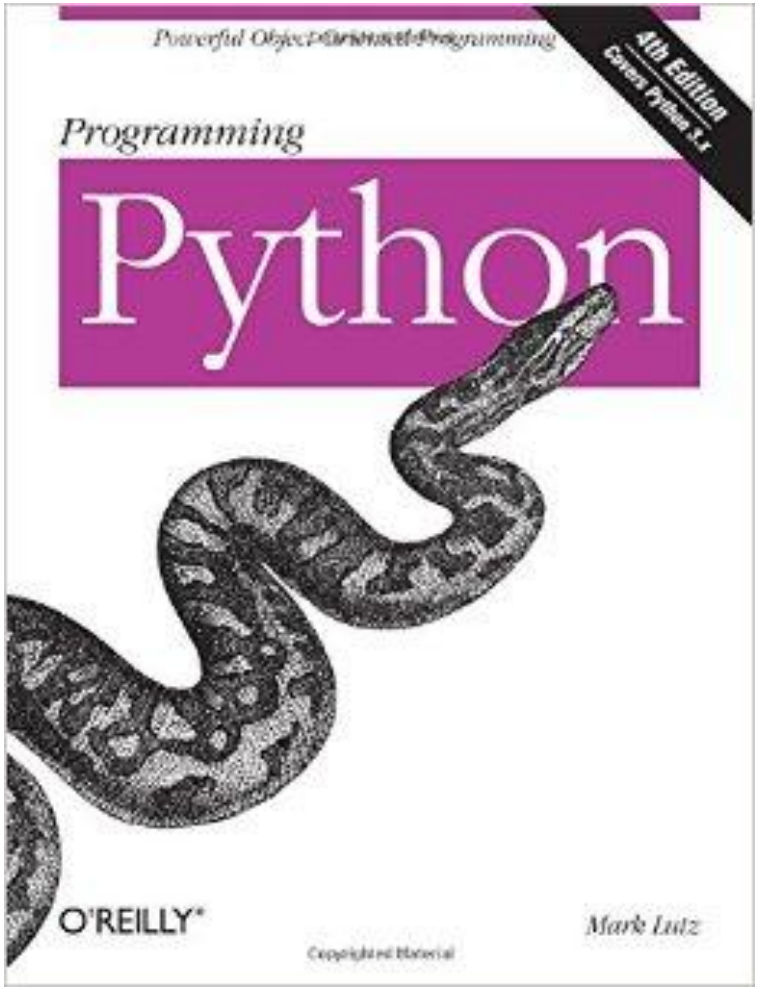
Text Book

Python Material (Level 2)



Text Book

Python Material (Level 3)



Text Book

Python Material



“Sololearn Python“

(A Simple Android

App for Beginners)

Online Materials:

Official Python Documentation:

https://docs.python.org/3/

Other Resources:

http://www.sololearn.com/Course/Python/

Popular Python recipes

http://code.activestate.com/recipes/langs/python/

Why Python?

Very easy to install and Setup

Easy to learn

Readable

Simplicity

Multipurpose

Errors appear on runtime. . . . . .



Reference: https://www.python.org/~guido/

Why Python..?

Software Quality

Python code is designed to be readable, and hence reusable and maintainable.

Productivity

Python code is typically one-third to one-fifth the size of equivalent C++ or Java code. That means there is less to type, less to debug, and less to maintain after the fact.

Portability

Most Python programs run unchanged on all major computer platforms. Porting Python code between Linux and Windows, for example, is usually just a matter of copying a script's code between machines.

Support Libraries

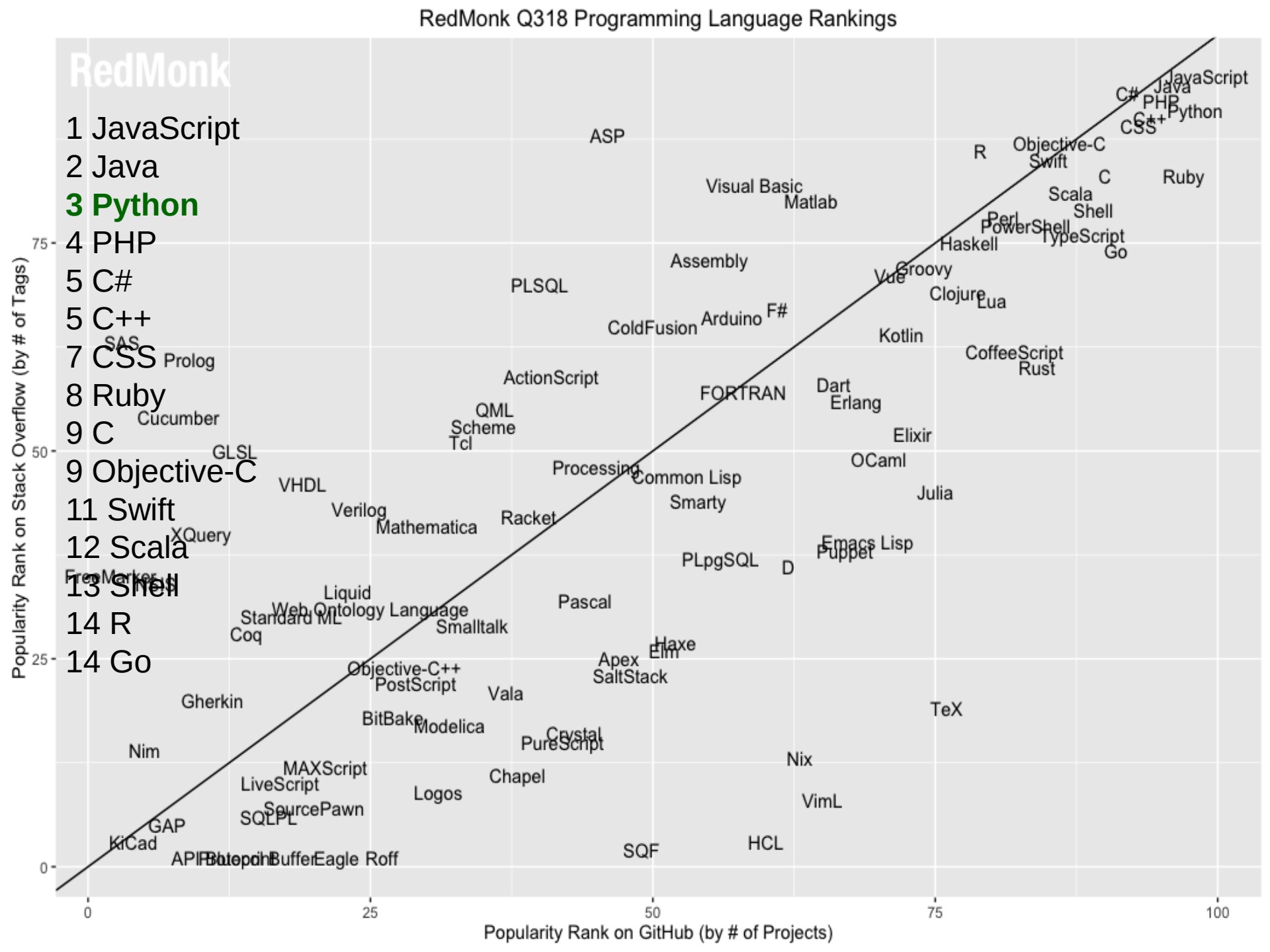
It has portable standard library, and many more free packages.

Component Integration

Python code can invoke C and C++ libraries, can be called from C and C++ programs, can integrate with Java and .NET components.

Enjoyment

Because of Python's ease of use and built-in toolset, it can make the act of programming more pleasure than chore.


RedMonk Q318 Programming Language Rankings
RedMonk
1 JavaScript
2 Java
3 Python
4 PHP
5 C#
5 C++
7 CSS
8 Ruby
9 C
9 Objective-C
11 Swift
12 Scala
13 Shell
14 R
14 Go
Popularity Rank on Stack Overflow (by # of Tags)
Popularity Rank on GitHub (by # of Projects)
0
25
50
75
100
JavaScript
Java
C#
PHP
C++
Python
CSS
Ruby
C
Objective-C
Swift
R
Scala
Shell
Perl
PowerShell
TypeScript
Go
Haskell
Groovy
Vue
Clojure
Lua
Kotlin
CoffeeScript
Rust
Dart
Erlang
Elixir
OCaml
Julia
Emacs Lisp
Puppet
D
TeX
Nix
VimL
HCL
SQF
Crystal
PureScript
Modelica
BitBake
Gherkin
Nim
MAXScript
LiveScript
SourcePawn
SQLPL
GAP
KiCad
Eagle
Roff
Logos
Chapel
Vala
Objective-C++
PostScript
Apex
SaltStack
Haxe
Elm
Pascal
Smalltalk
Web Ontology Language
Standard ML
Coq
Liquid
FreeMarker
XQuery
Mathematica
Racket
Verilog
VHDL
Processing
Common Lisp
Smarty
PLpgSQL
Cucumber
GLSL
QML
Scheme
Tcl
ActionScript
FORTRAN
SAS
Prolog
ColdFusion
Arduino
F#
PLSQL
Assembly
Visual Basic
Matlab
ASP

Comparison with other languages

- Python code is typically **3-5 times shorter than equivalent Java code**, it is often **5-10 times shorter than equivalent C++ code!**
- Anecdotal evidence suggests that one Python programmer can finish in two months what two C++ programmers can't complete in a year.
- Python shines as a glue language, used to combine components written in C++.

So, Python can increase productivity

Reference: https://www.python.org/doc/essays/comparisons/

Points to be noted

- **“Python is a scripting language"**
  - **False.** Python has been used as a scripting language, but it is also **used to develop large stand-alone applications.** (A general Purpose language)
- **Python is interpreted, thus slower than running native code**
  - **True, But not always**
  - Python can be used to `glue' together native modules.
  - Libraries are often very efficient.
- **Whitespaces are ugly**
  - You'll get used to it.
- **Dynamic typing is unsafe.**
  - Python is **strongly typed** and well behaved.
  - **It can deal with type errors at runtime**.

Reference: https://www.python.org/doc/essays/comparisons/

Application Areas

- Application Development
- Web Development
- Scripting
- Scientific Computing
- Machine Learning/Deep Learning

Success Stories: https://www.python.org/about/success/

Versions

- Python2
  - Python2 – Very Stable (Python-2.7 Mid 2010) – No release after that.
- Python3 (Python3.0 in 2008)
  - **Current Release – 3.7.0**
  - Some major changes and clean-ups
  - Not backward compatible (cannot execute 2.x code)
  - What is new in Python3? - https://docs.python.org/3/whatsnew/3.0.html
- We use Python3 in this course

So how do I get started?



Installing Python

- **Linux:** (Debian/Ubuntu)

  $ sudo apt-get install python3

- **Windows:**
  - Download from:

    https://www.python.org/downloads/
  - Click on the executable and install it.

Installing Anaconda

- Download the Anaconda installer for Linux from:
- In your terminal window type the following:

  bash Anaconda4-2.0.0-Linux-x86_64.sh

  NOTE: Replace ~/Downloads with your actual path and actual file name.
- Accept the default location or select a user-writable install location such as ~/anaconda.

  NOTE: Install Anaconda as a user unless root privileges are required.
- Follow the prompts on the installer screens, and if unsure about any setting, simply accept the defaults, as they can all be changed later.

  NOTE: If you select the option to not add the Anaconda directory to your bash shell PATH environment variable, you may later add this line to the .bashrc in your home directory:

  export PATH="/home/username/anaconda/bin:$PATH"
- Replace /home/username/anaconda with your actual path.
- Finally, close and re-open your terminal window for the changes to take effect.

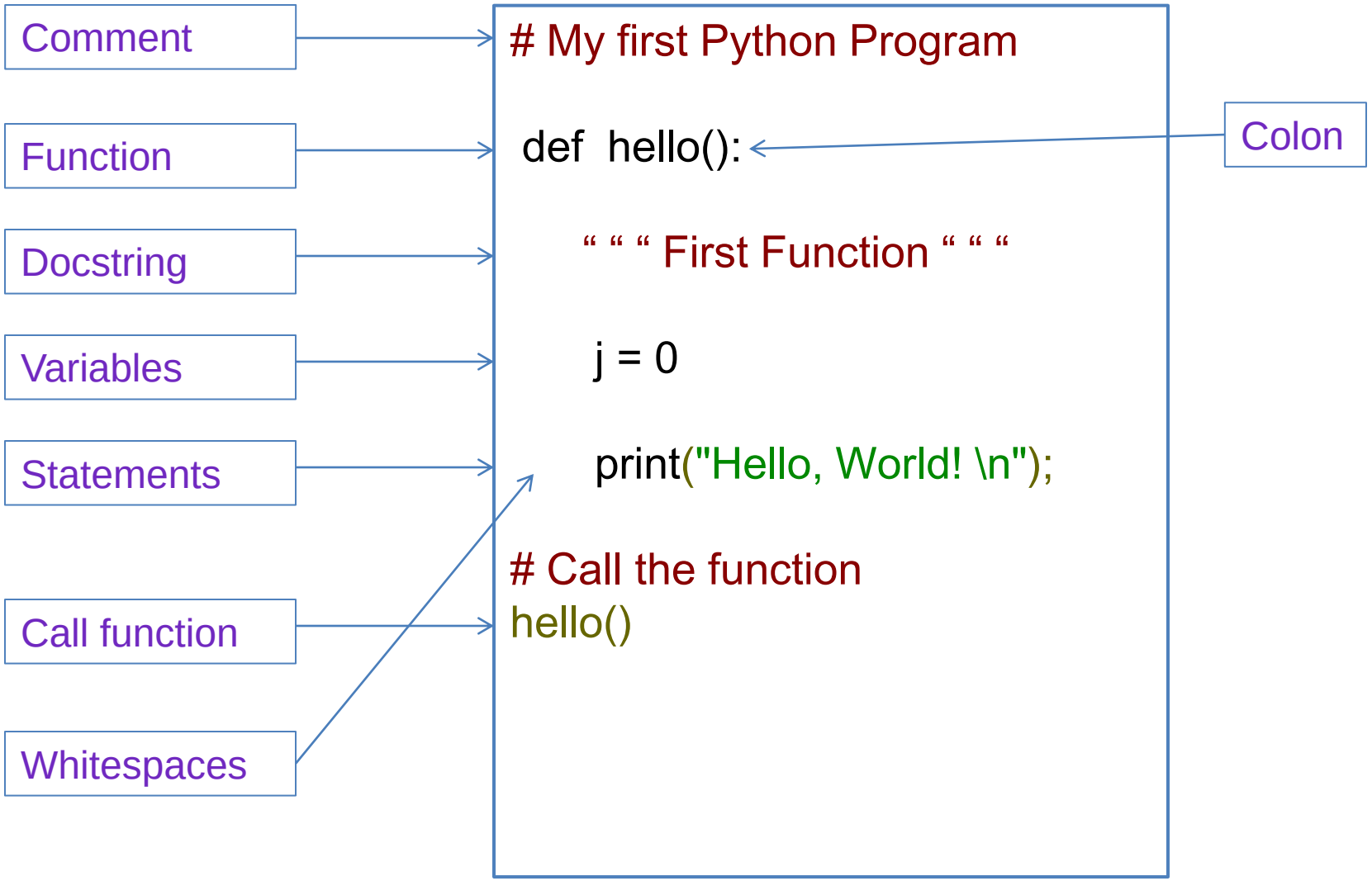
Installed Correctly???

Test using simple python statements such printing Hello World

```
$ python3
>>> print("Hey!! Am I installed")
```



A Simple Python Program – hello.py



Indentation

Python Blocks & Whitespaces

- Be cautious about indentation, blank spaces and line breaks
- Enforces readable code

Python

```
if x > 1:
<space-1>…<space-n> print x
<space-1>…<space-n> print x-1
```

C/C++/Java

```
if(x>1)
{
        printf("%d", x);
        printf("%d", x-1);
}
```

More on indentation..

Python Blocks & Whitespaces

- Be cautious about indentation - blank spaces and line breaks
- Enforces readable code

```
def perm(l):
        # Compute the list of all permutations of l
    if len(l) <= 1:
                  return [l]
    r = []
    for i in range(len(l)):
             s = l[:i] + l[i+1:]
             p = perm(s)
             for x in p:
              r.append(l[i:i+1] + x)
    return r
```

More on Indentation…

Note:

- **Do not use tabs** at all for indentation – Makes your code non-portable
- Use 3 or **4 spaces**
- Compiler ignores blank lines
- Line continuation character: \

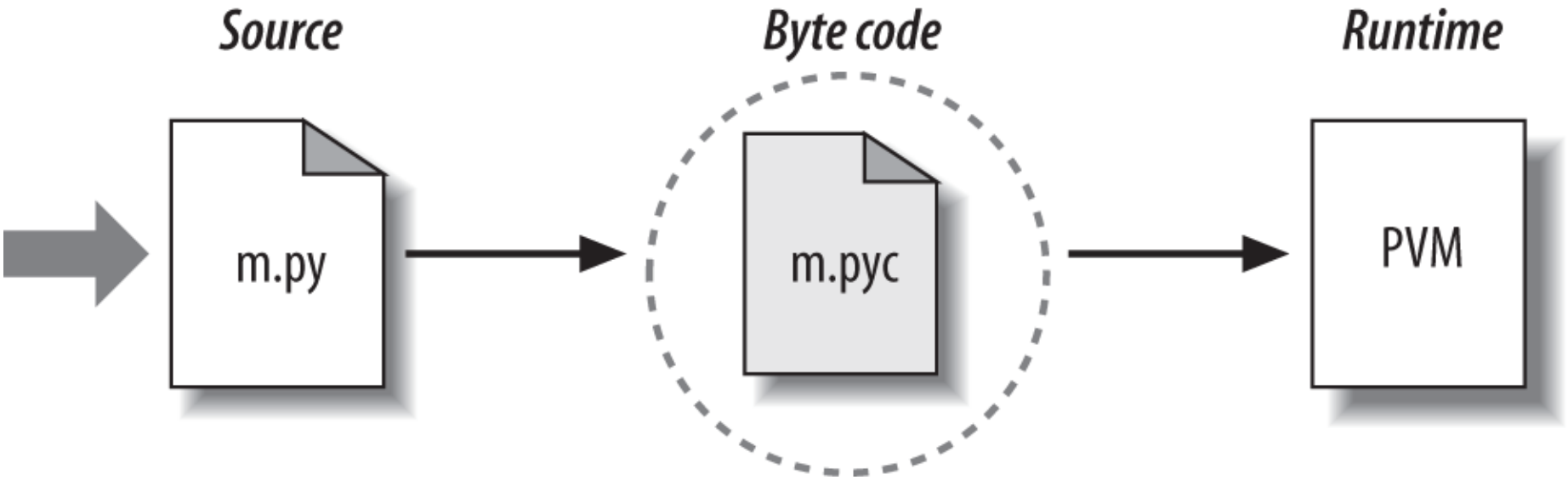
How to Run?

How to run?

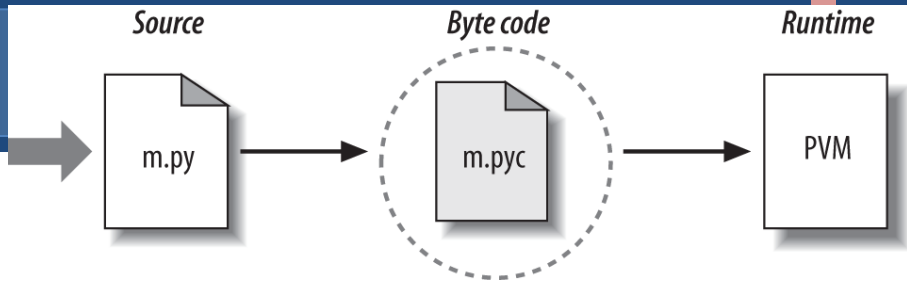
$ python3 hello.py

Output:

'Hello world !'



How to Run?



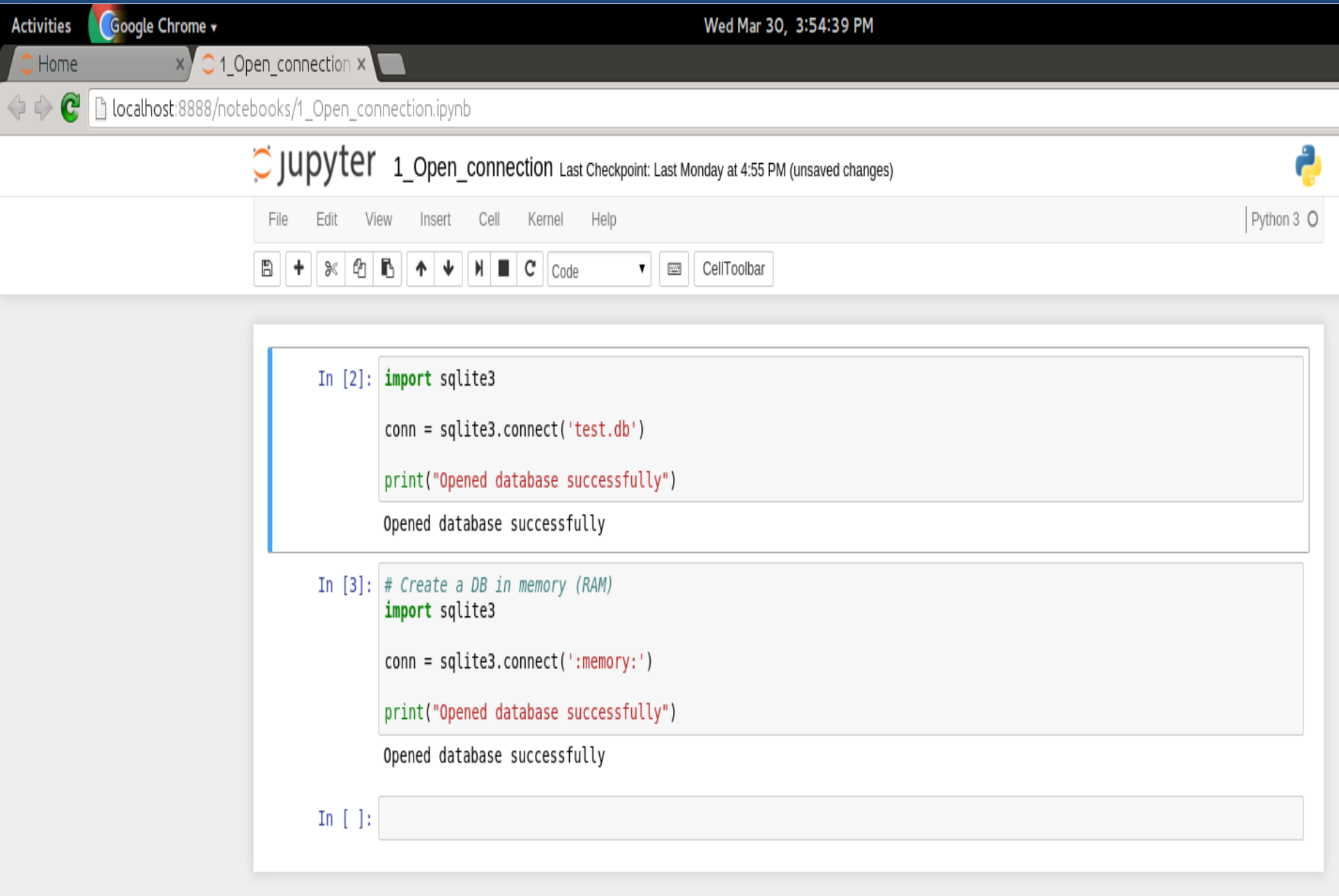
1. **Generate Byte Code**
   - Python Interpreter translates your source code into machine independent byte code (.pyc)
   - Stores .pyc file __PyCache__ folder
   - When you run the same program (Without changes) then it will use this byte code without translating it again.
2. **Run on PVM**
   - Byte Code (.pyc) will be shipped to PVM
   - It executes the code

Running Python – Interactive Mode

- Python interpreter can be run in an interactive session mode.
- Built-in `Read/Evaluate/Print-Loop' (REPL)
- Python statements are evaluated and the result is printed to the user.

- Python shells:
  - IDLE shell
  - **Ipython/Jupyter**
  - Eclipse
  - **vim**
  - nano

Jupyter


Activities
Google Chrome
Wed Mar 30, 3:54:39 PM
Home
1_Open_connection
localhost:8888/notebooks/1_Open_connection.ipynb
jupyter 1_Open_connection Last Checkpoint: Last Monday at 4:55 PM (unsaved changes)
File Edit View Insert Cell Kernel Help
Python 3
Code
CellToolbar
In [2]:
import sqlite3
conn = sqlite3.connect('test.db')
print("Opened database successfully")
Opened database successfully
In [3]:
Create a DB in memory (RAM)
import sqlite3
conn = sqlite3.connect(':memory:')
print("Opened database successfully")
Opened database successfully
In [ ]:

Converting notebook to script

```
$ jupyter-nbconvert file.ipynb --to script
```

It will create file.py

Basic Input and Output

Functions: input, print

1. input - help(input)

Usage: input(prompt='')

- method of ipykernel.ipkernel.IPythonKernel
- instance Forward raw_input to frontends
- Raises ------ StdinNotImplentedError if active frontend doesn't support stdin.

2. print - help(print)

Usage: print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

- Prints the values to a stream, or to sys.stdout by default, **returns None**
- Optional keyword arguments:
  - **file:** a file-like object (stream); defaults to the current sys.stdout.
  - **sep:** string inserted between values, default a space.
  - **end:** string appended after the last value, default a newline.
  - **flush:** whether to forcibly flush the stream.

Basic Input and Output - Examples

How to use input, print?

```
>>>name = input('What is your name?:')
What is your name?:Ram
>>>print(name)
Ram
```

- All input is returned by the input function as a string type.
- For the input of numeric values, the response must be converted to the appropriate type.
- Python provides built-in type conversion functions **int( )** and **float( )** for this purpose,

```
>>> salary = input('What is your salary?:')
What is your salary?:94532.50
>>>type(salary)
<class 'str'>
>>> salary = float(salary)
>>> type(salary)
```

Strong and Dynamic types

Python is both strongly typed and dynamically typed

- **Strong** typing: The type of a value doesn't suddenly change. A string containing only digits doesn't magically become a number, as may happen in Perl. Every change of type requires an explicit conversion.

- **Dynamic** typing means that runtime objects (values) have a type, as opposed to static typing (Like C variables) where variables have a type.

```
i = 10
s = "BLR"
print("Value of i+s:", i+s) #Error, i and s are strongly typed, we cannot add

# Change the type of s dynamically, from string to int
s = 20
print("Value of i+s:", i+s)
```

Strong Typing

- Operations may expect operands of certain types.
- Interpreter throws an exception if type is invalid.

```
>>> a = 5
>>> b = 'Python'
>>> a + b
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int'
and 'str'
```

Dynamic Typing

- Type checks (making sure variables have the correct type for an operation) performed at runtime.
- No need to declare variable types.
- We can obtain the type of an object with `type(variable)‘.

```
>>> x = 5*4
# create a new string object and let x point to it.
>>> x = 'twenty'
>>> x
'twenty'
>>> type(x)
<type 'str '>
```

Coercion

- **Coercion** is the automatic conversion of operands to a common type.

```
>>> 1.2 + 4
5.2  (4 is converted to float automatically)
```

- Type conversion is the **explicit conversion** of an operand to a desired type.
  - Functions: int( ), float( )

```
>>> int(10.4)
10
>>> float(4)
4.0
```

Comments & Docstrings

Single Line Comments:

- Single line comment starts with # at the end of a line.

```
# Print a welcome messages.
print ('Hello world !') # Be friendly…write comments
```

Doc Strings:

- Used to describe a function, module, method, class etc.
- Used for code documentation
- **These are interpreted**, use if necessary.

```
def sum(a, b):
    """ Compute the sum of two integers
    and return to the calling function """
    return a+b
sum.__doc__    # Print docstring of the function sum
```

Joining Lines

Implicit:

- Allow a logical program line to span more than one physical line using certain matching delimiting characters – (), {}, [], “”””.

```
print('Name:', student_name, 'Address:', address,
'Number of Credits:', total_credits, 'GPA:', gpa)
```

Explicit:

- Explicitly joined by use of the backslash (\) character.

```
Avg = math_marks + science_marks + Hindi_marks + \
      english_marks + socail_marks
```

Good Programming Practices

- Do not use semicolons, they are legal but unnecessary.

```
x = 5

x = 5;
```

Both are correct.

- Limit lines to 79 characters
- Python is case sensitive
  - All key words are case sensitive
  - Class Name should be written in CamleCase
  - Every thing else should be in lower case (Underscore is acceptable)
  - For more details see **PEP8 (Python Enhanced Proposals – 8).**

Literals

- A **literal** is a sequence of one or more characters that stands for itself.
- A **numeric literal** is a literal containing only the digits 0–9, an optional sign character (1 or 2), and a possible decimal point.
  - Commas are never used in numeric literals.

int literals	float literals				Incorrect
5	5.	5.0	5.125	0.005	5,000.439
250	250.	250.0	250.12		2,500
+243	+243.	+243.0	+243.124		+2,500.43
-210	-210.	-210.0	-210.326		-2,500.43

Exercise

- Print the following literals in the python shell and observe the results.

int literals	float literals			Incorrect
5	5.	5.0	5.125 0.005	5,000.439
250	250.	250.0	250.12	2,500
+243	+243.	+243.0	+243.124	+2,500.43
-210	-210.	-210.0	-210.326	-2,500.43

It will take +2,500.43 as → +2 as one const and 500.43 as second const since comma separates two consts or vars

String Literals

- **String literals**, or “strings” represent a sequence of characters
  - Delimit using single or double quotes**.**
  - Multi line strings are allowed (doc strings)
  - Includes letters, digits, special characters and blanks
  - Blank and Empty strings are allowed (“ “, “”)
- **Character representation**
  - **Uses Unicode encoding** (Compatible with ASCII).
  - Unicode can represent 4 billion characters.
  - **Examples:** ‘A’ – 65, ‘B’ – 66, ‘0’ - 48
  - **ord function – ordinal of a char**
    - >>> **ord(‘A’)**

      65

Characters	ASCII Values
A – Z	65 – 90
a – z	97 – 122
0 – 9	48 – 57
special symbols	0 - 47, 58 - 64, 91 - 96, 123 - 127

Constants

- Fixed values such as numbers, letters, and strings are called “constants”
- Value does not change
- String constants either use single quotes (‘) or double quotes (“)

```
>>> print(12)
12
>>> print(12.45)
12.45
>>> print('Hello world')
Hello world
```

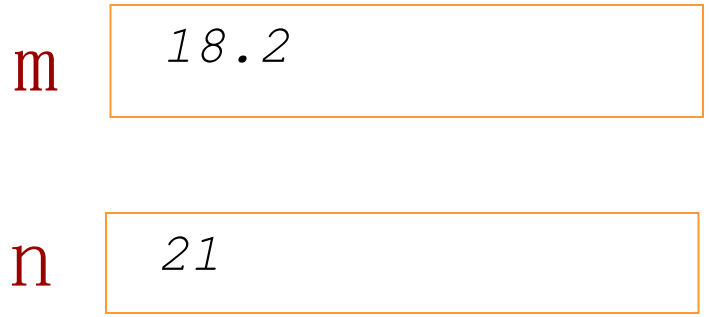
Variables and Assignment

- A variable is a **named place** in the memory.
- A programmer can store data and later retrieve the data using the variable "name".
- Programmers get to choose the names of the variables.
- You can change the contents of a variable in a later statement.

```
m = 18.2
n = 21
```

(OR)

```
m, n = 18.2, 21
```

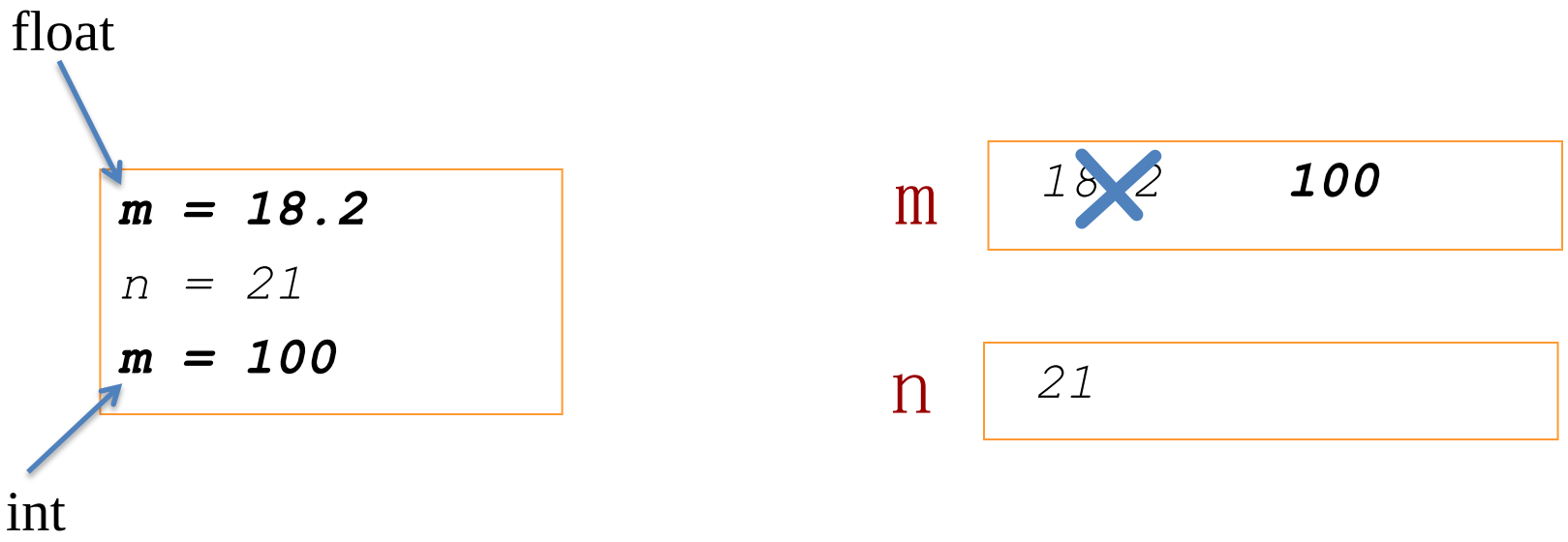


Swap

```
m, n = n, m
```

Variables and Assignment…

- You can change the contents of a variable in a later statement



In Python the same variable can be **associated with values of different type** during program execution

Let us look at C-Style variables

```
/* c code */
int a = 5;
int b = 5;
```

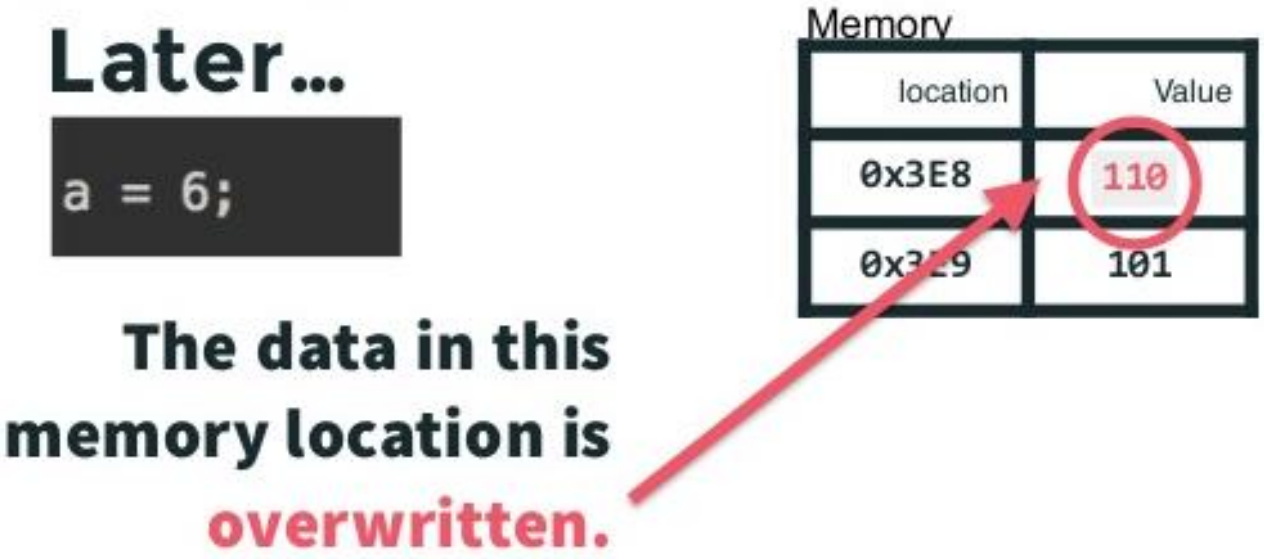
Memory

variable	location	Value
a	0x3E8	101
b	0x3E9	101

These values live in a fixed size bucket.

Can only hold same-sized data, or an overflow occurs.

Let us look at C-Style variables



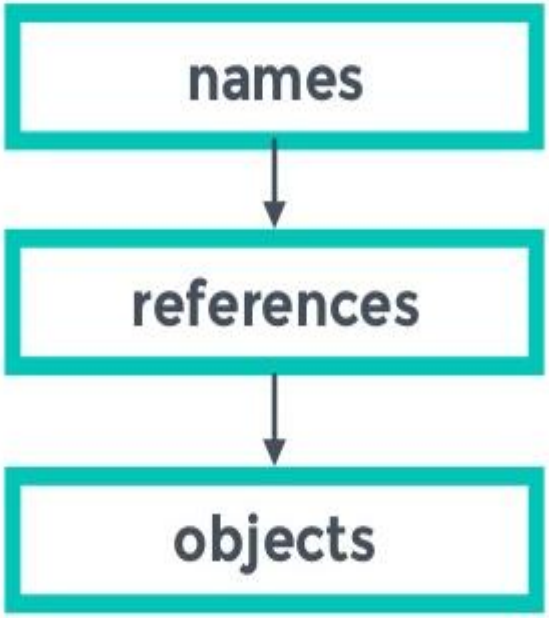
Value of “a” is changed from “5” to “6”, in the same location it is changed…

What about Python?

“Python has names, not variables”

What it means?

How python objects are stored in Memory?



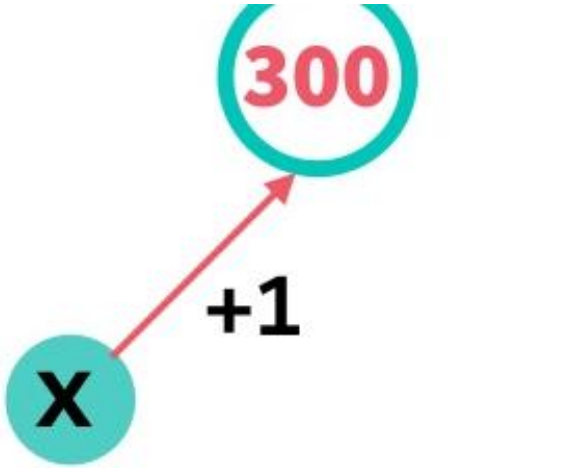
A name is just a label given to an object, an object can have many names.

Number of references will be the number of labels given to an object

An object can be “a number’ or “a string” or “a list” or “ a function” ….so on

Understanding Reference Count

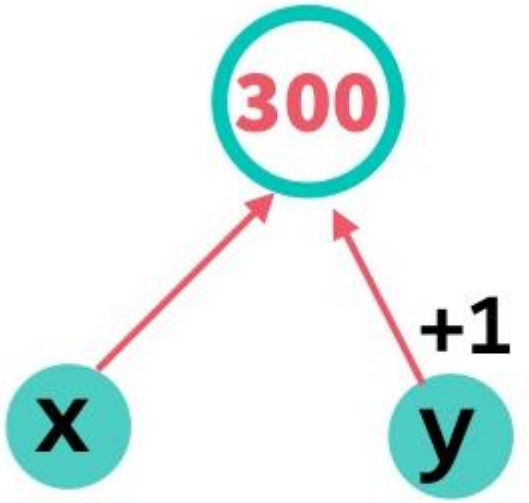
```
x = 300
```



references: 1

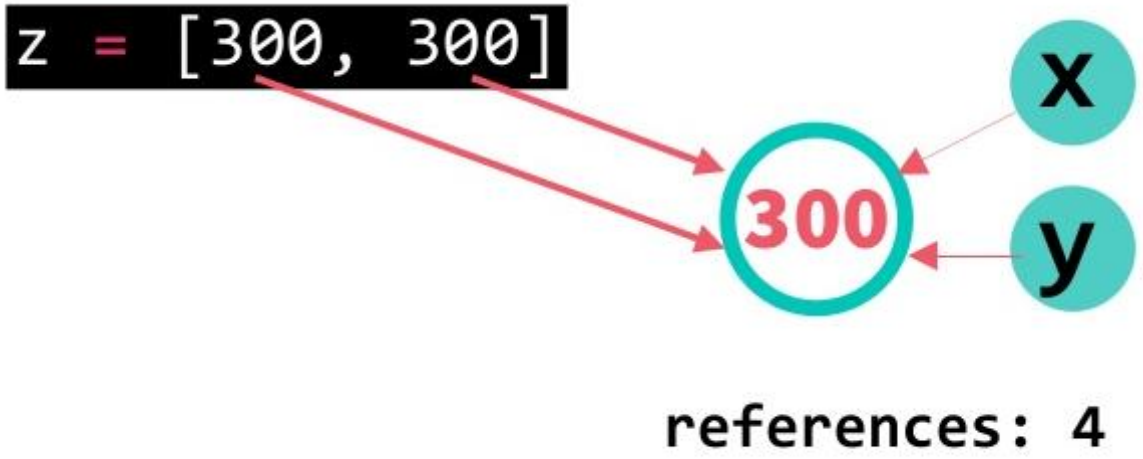
Understanding Reference Count…

```
x = 300
y = 300
```



There are two labels to the object “300”,
Reference count: 2

Understanding Reference Count…

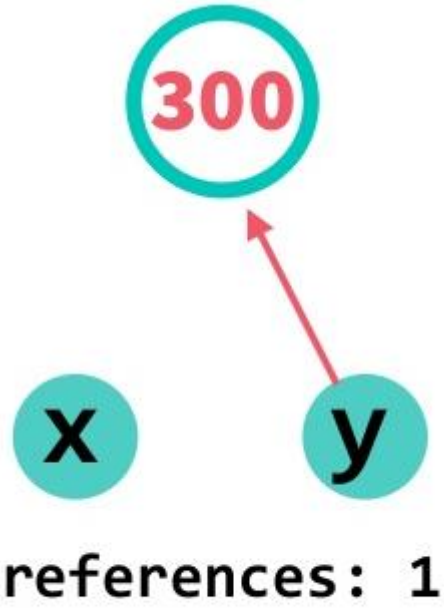


There are 4 labels to the object “300”,
Reference count : 4

Understanding Reference Count…

```
x = 300
y = 300

del x
```



Decrease reference count by 1

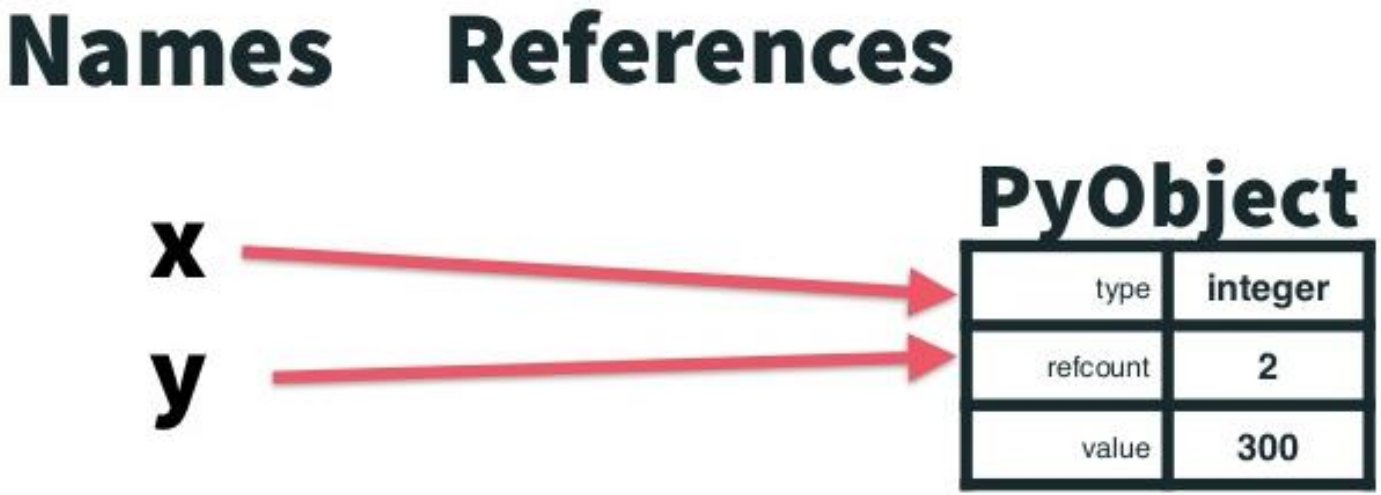
"del" doesn't delete the object, instead it deletes the label to that object and reduces the ref count by 1

If it is zero then, it will do garbage collection, deletes the object

A python object

Every python object holds 3 things:

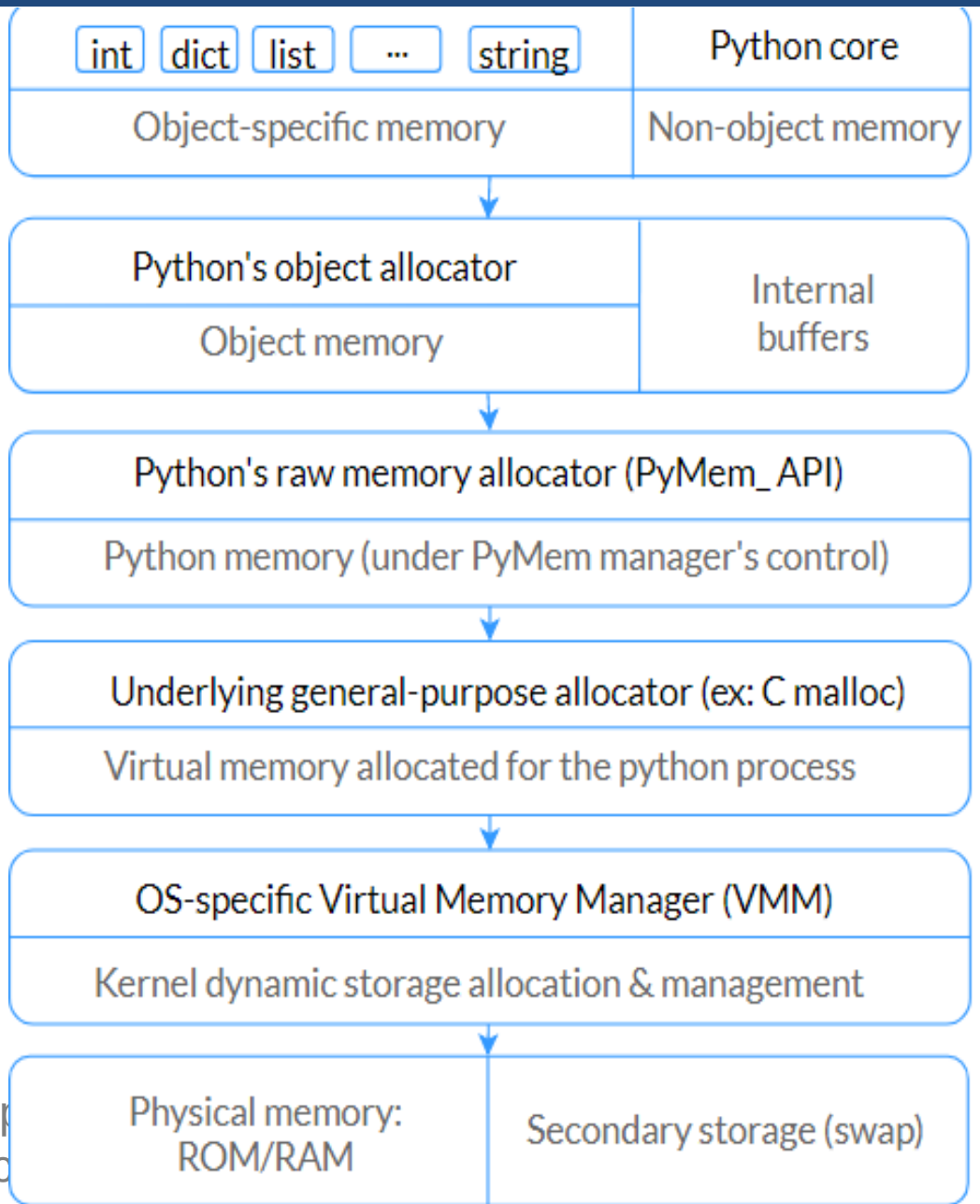
- Its type
- Value
- A reference count



More details on Memory Management

How Python manages its memory?

Illustration of memory hierarchy



Ref:
https://github.com/p 506c09bc2a544247
4/Objects/obmalloc

Python memory allocation

Every thing in Python is an "Object"

To speed-up memory operations and reduce fragmentation Python uses a special manager on top of the general-purpose allocator, called **"PyMalloc"**

There are 2 Types of allocation:

- **Big objects** are allocated by standard memory allocation
- **Small objects** are allocated in three levels of abstraction
  - Block
  - Pool
  - Arena

Small object allocation – (a) Block

Block:

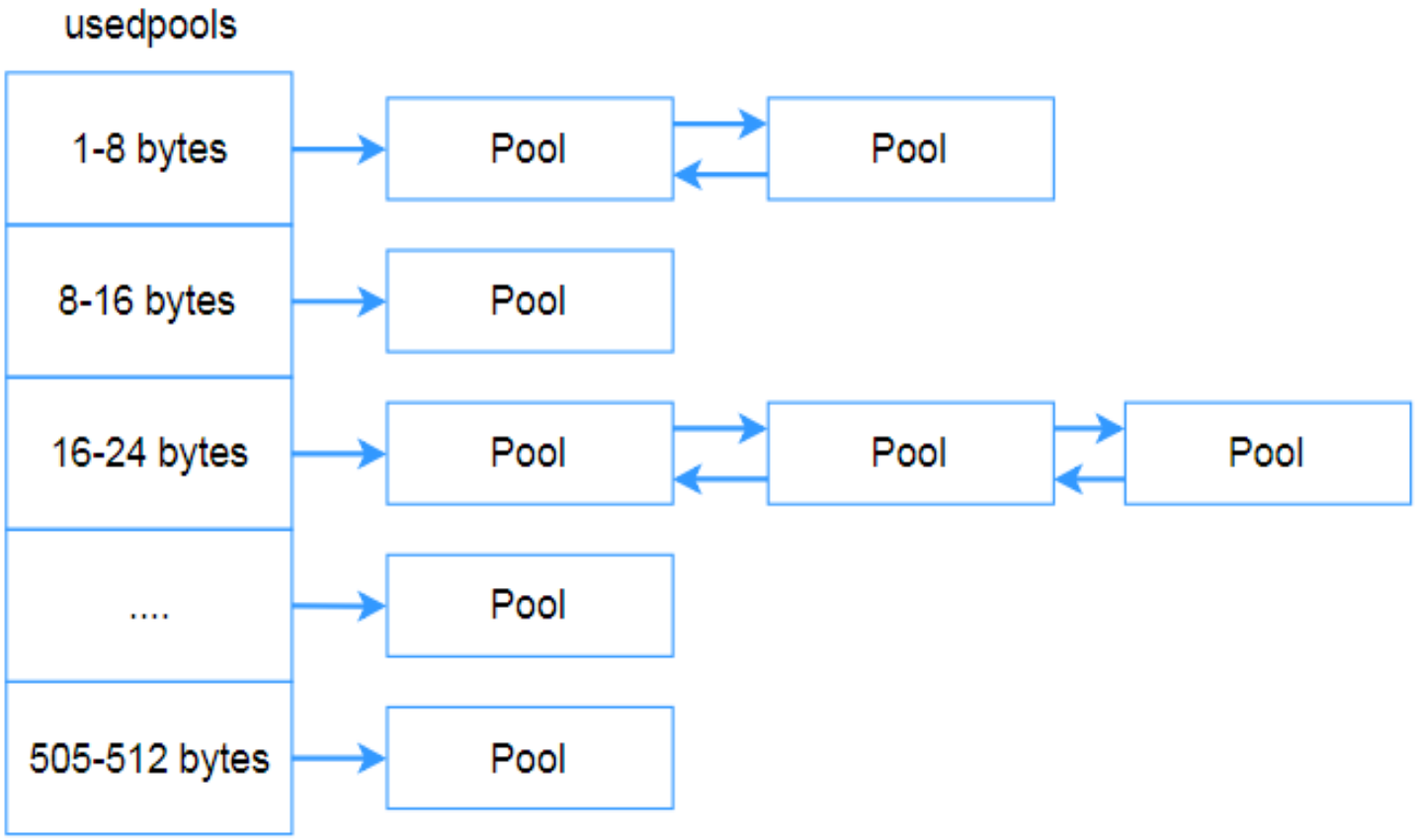
- Block is a chunk of memory of a certain size.
- Each block can keep only one Python object of a fixed size.
- The size of the block can vary from 8 to 512 bytes

Request in bytes	Size of allocated block
1-8	8
9-16	16
17-24	24
25-32	32
33-40	40
41-48	48
...	...
505-512	512

Small object allocation – (b) Pool

Pool:

- A collection of blocks of the same size is called a pool.
- Normally, the size of the pool is equal to the size of a memory page, i.e., 4Kb.



Small object allocation – (c) Arena

Arena:

- The arena is a chunk of 256kB memory allocated on the heap, which provides memory for 64 pools.

Arena

Pool (4kB)	Pool (4kB)	Pool (4kB)
Pool (4kB)	Pool (4kB)	Pool (4kB)
Pool (4kB)	Pool (4kB)	Pool (4kB)
Pool (4kB)	Pool (4kB)	Free Pool (4kB)
Free Pool (4kB)	Free Pool (4kB)	Free Pool (4kB)
...	...	...

More details on Memory Management

Take an example..
How it manages small objects efficiently?

Storing a small integer object

A small object will have only one copy in memory and all the labels/variables having this value will refer to this memory location.

```
>> a = 10
>> b = 10
>> c = 10
>> print(id(a), id(b), id(c))

621897573616, 621897573616, 621897573616

>> a += 1

>> id(a)
621897573592
```

Storing a small integer object…

```
>> a = 500
>> b = 500
>> print(id(a), id(b))
```

621897573620, 621897573408

What happened?
Why different addresses/IDs?

Python keeps an array of integer objects for all integers between -5 and 256.

So when we create an integer in that range, they simply back reference to the existing object.

Variable Rules

- Must start with a letter or underscore _
- Must consist of letters and numbers and underscores
- Case sensitive

Examples:

```
Good:       dsbda   pi      pi34    _speed
Bad:        23spam  #sign   var.12
```

Reserved Words

```
False      class      finally    is         return
None       continue   for        lambda     try
True       def        from       nonlocal   while
and        del        global     not        with
as         elif       if         or         yield
assert     else       import     pass
break      except     in         raise
```

Operators, operands and expressions

- Operators are symbols which take one or more operands or expressions and perform arithmetic or logical computations.

  Examples: +, -, *, /, =, ==

- An "operand" is an entity on which an operator acts.

  Example: a+b-5 → a, b and 5 are operands

- An "expression" is a sequence of operators and operands that performs any combination of these actions:
  - Computes a value
  - Designates an object or function
  - Generates side effects

  Example: a + (b / 6.0)

Operators

Python language supports the following types of operators:

Type	Operators	Examples
Arithmetic Operators	+, -, *, / Modulus - % Exponent - ** Flat Division - //	21 % 10 = 1 21 // 10 = 2 (Fraction is omitted) 21.0 // 10.0 = 21.0/10 = 21/10.0 = 2.0
Comparison (Relational) Operators	==, !=, <, >, >=, <= < > (Python2 only)	20 != 10 O/P: True
Assignment Operators	=, +=, -+, *=, /+, %=, **=, //=	a=24 a//=10 → a = a//10
Logical Operators	and, or, not	(10 and 20)
Bitwise Operators	&, |, ^ (XOR), <<, >> ~ (1's complement),	~(2) = -3
Membership Operators	in not in	x in y → Return 1 if x is a member of sequence y.
Identity Operators	is is not	x is y → True if x and y point to the same object

Operator Precedence

tuple, list, dictionary, string	(), [], {}, ‘’
attribute, index, slide, function call	x.attr, x[], x[i:j], f()
Exponentiation	**
Bitwise not	~x
Positive and negative	+x, -x
Multiplication, division, remainder	*, /, %
Addition, subtraction	+, -
Bitwise shift	<<, >>
Bitwise AND	&
Bitwise XOR	^
Bitwise OR	|
Comparisons, membership, identity	<, <=, >, >=, <>, !=, ==, in, not in, is, is not
Boolean Not	not x
Boolean AND	and
Boolean OR	or

High

Low

Data Types

- Elementary Types
  - **NoneType**: None
  - **bool**: True, False
  - **int**: 42
  - **float**: 3.14, complex: (0.3+2j)
- Container or Sequence Types
  - **str**: 'Hello‘ (Sequence of Unicode characters)
  - **list**: [1, 2, 3]
  - **tuple**: (1, 2, 3)
  - **dict**: {'A':1, 'B':2}
  - **set**: {1, 2, 3}
- file
- function, class, instance

Note: In Python everything is an object and every object has a type.

Range of int and float

- **int:** There is no limit on integer size (No standard followed)

```
# Assign a large integer to a variable and print it.
x = 346523842342342432424
print(x) # It will print 346523842342342432424
```

- **float:** float has limited **range** and limited **precision**
  - **Uses IEEE-754** double precision format
  - Range: 10^308 – 10^-308
  - Precision: 16 to 17 digits

```
# Assign a large integer to a variable and print it.
x=9.546789045621234567 8 # 19 digits
print(x) # OP: 9.546789045621235 i.e. 15 digits
```

Arithmetic Overflow and Underflow

- **Overflow:** Calculated result is too large in magnitude (size) to be represented.

```
>>>1.2e200 * 2.1e210
inf
```

- **Underflow:** Calculated result is too small in magnitude (size) to be represented.
  - Division of two numbers may result in underflow

```
>>>1.0e-300 / 1.0e100
0.0
```

Mutable and Immutable

- **Mutable objects can be modified - lists, dictionaries, sets**

```
>>> cats = ['tommy', 'lucky ', 'spot']
# Add an element to the list object
>>> cats.append ('Bond')
>>> cats
['tommy', 'lucky ', 'spot', 'Bond']
```

- **Immutable objects cannot be changed after initialization – Boolean, numbers, strings, tuples**

```
>>> s = "cat"
# Try to change the one character of string s
>>> s[0] = 'm'
TypeError: str object does not support item assignment
```

Mutable and Immutable…

- **Immutable objects cannot be changed after initialization – Boolean, numbers, strings, tuples**

- **Can we reinitialize string with another string???**

```
>>> s = "cat"
# Initialize cat with another name – It creates another object Bond with the name cat, we are not modifying the string,  a new string is created with the same name
>>> cat = 'Bond'
>>> cat
Bond
```

Class Room Exercise

1. Read two numbers from the user and swap them without using third variable.
2. Solve a quadratic equation ax**2 + bx + c = 0. Ask the user to give the values of the coefficients a, b and c.

Hint: It will have two solutions.

solution1 = [-b - sqrt((b**2) - (4*a*c))] / 2*a

solution2 = [-b + sqrt((b**2) - (4*a*c))] / 2*a

Lab Exercises

1. Use the “format” function to display the following floating-point values with three decimal digits of precision.
(a) 4580.5034 (b) 0.00000046004 (c) 5000402.000000000006

2. Give a call to print that is provided one string that displays the following address on three separate lines.

CDAC
Old Madras Road
Bangalore

3. What is the value of variables num1 and num2 after the following instructions are executed?

```
num = 5
k = 5
num1 = num + k * 2
num2 = num + k * 2
```

Are the values id(num1) and id(num2) equal after the last statement is executed?

4. Evaluate the following expressions. (Use operator precedence and associativity).

Let var1 = 10, var2 = 30 and var3 = 2.

(a) var1 - 6 ** 4 * var2 ** 3 (b) var1 // var2 / var3 (c) var1 * var2 / 4 // var3

Lab Exercises…

5. Write a Python program that prompts the user to enter an upper or lower case letter and displays the corresponding Unicode encoding.

6. Develop and test a program that prompts the user for their age and determines approximately how many breaths and how many heartbeats the person has had in their life. The average respiration (breath) rate of people changes during different stages of development. Use the breath rates given below for use in your program:

	Breaths per Minute
Infant	30–60
1–4 years	20–30
5–14 years	15–25
Adults	12–20

For heart rate, use an average of 67.5 beats per second.

Summary

- ✓Introduction to Python
- ✓A simple program
- ✓Python Objects and Labels
- ✓Data types - Strict and dynamic Types
- ✓Memory Management
- ✓Operators and Expressions
- ✓Mutable and Immutable Objects

Thank You